

Задание

Разработать программу для обработки пассажиров железнодорожного состава

Поезд состоит из вагонов. Каждый вагон хранит собственный список пассажиров. Для части операций нужен единый логический проход по всем пассажирам поезда, хотя физически одного общего контейнера пассажиров нет

Модель данных:

```
Train
  Wagon
    Passenger
    Passenger
  Wagon
    Passenger
```

Путь к исходному файлу передается через аргумент командной строки

Пример запуска:

```
./train_passengers train.txt
```

Файл содержит описание вагонов и пассажиров:

```
WAGON 1 COUPE
P Alexey Orlov 12 Vladivostok
P Maria Kim 13 Khabarovsk
WAGON 2 PLATZKART
P Ivan Petrov 31 Ussuriysk
P Anna Lee 32 Vladivostok
```

Строка вагона имеет формат:

```
WAGON номер тип
```

Строка пассажира имеет формат:

```
P имя фамилия место станция_назначения
```

Пассажир относится к последнему объявленному вагону. Формат строк можно считать корректным

Для вагона необходимо хранить:

- номер вагона
- тип вагона
- коллекцию пассажиров

Для пассажира необходимо хранить:

- имя
- фамилию
- номер места
- станцию назначения

Нужно реализовать пользовательский range для прохода по пассажирам всего поезда:

```
for (const auto &passenger : train.passengers()) {  
    // обработка пассажира  
}
```

Range должен проходить по структуре Train -> Wagon -> Passenger и скрывать от клиентского кода вложенное хранение данных

Iterator должен поддерживать:

- dereference текущего пассажира
- переход к следующему пассажиру
- сравнение с end iterator
- работу с const Train

Программа должна использовать созданный range для следующих операций:

- вывести всех пассажиров в порядке вагонов и мест
- найти пассажира по фамилии
- посчитать количество пассажиров до станции Vladivostok
- построить статистику количества пассажиров по станциям назначения
- найти первого пассажира с номером места больше 30
- вывести список фамилий пассажиров через range algorithm с projection

Для поиска и подсчета использовать `std::ranges algorithms`. Projection должен использоваться хотя бы в одной операции, например при поиске по фамилии или сортировке копии фамилий

Дополнительно можно использовать `std::views::filter` и `std::views::transform` для вывода пассажиров, следующих до выбранной станции. Эта часть не является обязательной

Программа должна сохранить итоговый отчет в файл `train_report.txt`

Отчет должен содержать:

- количество вагонов
- общее количество пассажиров
- список всех пассажиров
- результат поиска пассажира по фамилии Kim
- количество пассажиров до станции Vladivostok
- статистику по станциям назначения
- результат поиска первого пассажира с местом больше 30
- список фамилий пассажиров

Интерактивный CLI реализовывать не требуется. Достаточно обработать аргумент командной строки, прочитать файл и сформировать отчет

Требования

- Названия переменных, функций, типов и других сущностей программы должны быть осмысленными и записываться на английском языке без транслитерации
- Программа должна читать путь к входному файлу из аргументов командной строки
- Программа должна обрабатывать запуск без аргумента командной строки
- В программе должны быть типы Train, Wagon и Passenger
- Каждый Wagon должен хранить собственную коллекцию пассажиров
- В программе должен быть реализован пользовательский range для прохода по всем пассажирам поезда

- Iterator пользовательского range должен поддерживать operator*, operator+, begin и end
- Range должен работать для const Train
- Программа должна использовать std::ranges algorithms для поиска или подсчета пассажиров
- В программе должна быть хотя бы одна операция с projection
- std::views::filter и std::views::transform можно использовать как дополнительную часть
- Программа должна сохранять итоговый отчет в файл train_report.txt
- Программа должна быть грамотно разбита на модули и содержать как минимум один .hpp файл и два .cpp файла. Допускается использовать больше файлов
- Для заголовочных файлов использовать #pragma once
- Программа должна компилироваться и выполняться без ошибок
- Студент должен понимать написанный код, уметь обосновать свое решение и отвечать на вопросы во время защиты лабораторной работы

Минимальный набор операций:

- чтение поезда из входного потока
- получение range всех пассажиров поезда
- переход iterator к следующему пассажиру
- поиск пассажира через std::ranges
- подсчет пассажиров через std::ranges
- запись итогового отчета в выходной поток